

Nama : Rafly Maulana Yusuf
NPM : 242310014
Kelas : TI-24-KA
Mata Kuliah : Lab. Pemrograman Web

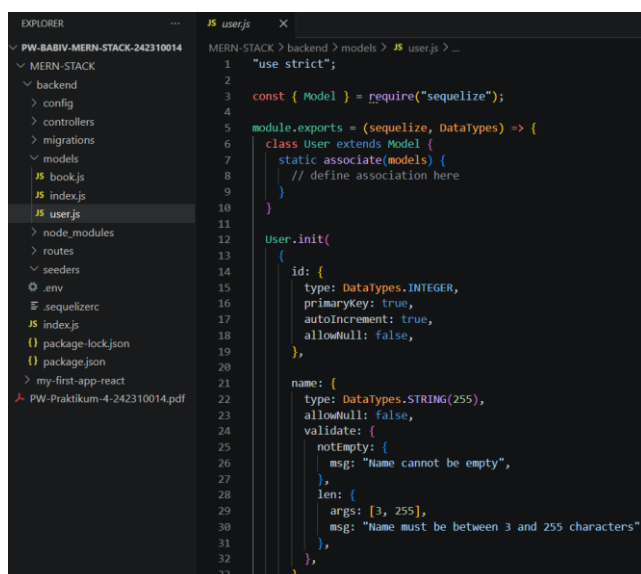
BAB VII (MEMBANGUN REST API DENGAN EXPRESS.JS)

1. Tujuan

Pada tugas pembelajaran ini dilakukan pengembangan backend menggunakan **Node.js**, **Express.js**, dan **Sequelize ORM** untuk membangun modul **User Management**. Modul ini bertujuan untuk mengelola data pengguna melalui REST API sehingga aplikasi dapat melakukan operasi Create, Read, Update, dan Delete (CRUD) terhadap data pengguna. Pengembangan dilakukan menggunakan migration, model, controller, dan routing sesuai dengan arsitektur Express.

2. Pembuatan Model User

Langkah pertama adalah membuat model **User** menggunakan Sequelize CLI. Model ini berfungsi sebagai representasi dari tabel **users** pada database MySQL. Selain mendefinisikan struktur data, model juga dilengkapi dengan validasi seperti nama tidak boleh kosong, email harus memiliki format yang benar, dan password memiliki panjang minimal enam karakter.



Penjelasan

Model User digunakan sebagai penghubung antara aplikasi Express dengan tabel **users** pada database. Dengan adanya model ini, proses CRUD dapat dilakukan menggunakan Sequelize tanpa perlu menulis query SQL secara langsung.

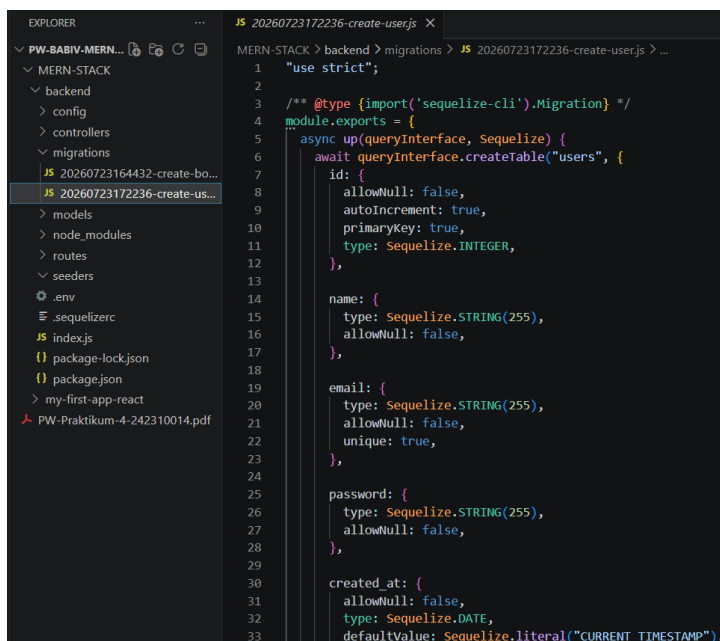
3. Pembuatan Migration

Setelah model dibuat, langkah berikutnya adalah membuat migration yang berfungsi membangun tabel **users** beserta atributnya.

Field yang digunakan meliputi:

- id
- name
- email
- password
- created_at
- updated_at

Selain itu ditambahkan index pada kolom **email** dan **name** agar proses pencarian data menjadi lebih cepat.



```
1  "use strict";
2
3  /** @type {import('sequelize-cli').Migration} */
4  module.exports = {
5    async up(queryInterface, Sequelize) {
6      await queryInterface.createTable("users", {
7        id: {
8          allowNull: false,
9          autoIncrement: true,
10         primaryKey: true,
11         type: Sequelize.INTEGER,
12       },
13
14       name: {
15         type: Sequelize.STRING(255),
16         allowNull: false,
17       },
18
19       email: {
20         type: Sequelize.STRING(255),
21         allowNull: false,
22         unique: true,
23       },
24
25       password: {
26         type: Sequelize.STRING(255),
27         allowNull: false,
28       },
29
30       created_at: {
31         allowNull: false,
32         type: Sequelize.DATE,
33         defaultValue: Sequelize.literal("CURRENT_TIMESTAMP"),
```

Penjelasan

Migration digunakan untuk mendefinisikan struktur tabel pada database sehingga proses pembuatan maupun perubahan struktur database dapat dilakukan secara otomatis menggunakan Sequelize CLI.

4. Menjalankan Migration

Migration dijalankan menggunakan perintah:

```
npx sequelize-cli db:migrate
```

Setelah proses berhasil, Sequelize akan membuat tabel **users** pada database **pw_praktikum_db**.

```
PS D:\College\Semester 4\Lab. Pemrograman Web - ASDOS\Praktikum Lab PW\PW-BABIV-MERN-STACK-242310014\MERN-STACK\backend> npx sequelize-cli db:migrate

Sequelize CLI [Node: 24.14.1, CLI: 6.6.5, ORM: 6.37.8]

◇ injected env (7) from .env // tip: % enable debugging { debug: true }
Loaded configuration file "config/config.js".
Using environment "production".
== 20260723172236-create-user: migrating =====
== 20260723172236-create-user: migrated (0.087s)
```

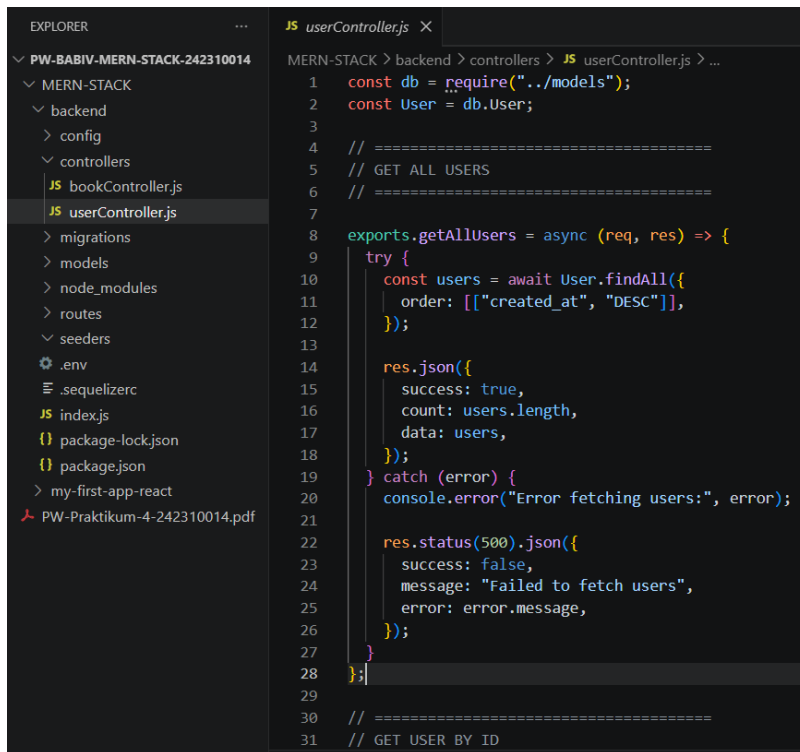
Penjelasan

Proses migration berhasil dijalankan tanpa error sehingga tabel **users** berhasil dibuat pada database sesuai struktur yang telah didefinisikan sebelumnya.

5. Implementasi Controller

Selanjutnya dibuat file **UserController.js** yang berisi seluruh logika bisnis untuk mengelola data pengguna. Controller terdiri dari beberapa fungsi yaitu:

- `getAllUsers()`
- `getUserById()`
- `createUser()`
- `updateUser()`
- `deleteUser()`



```
1  const db = require("../models");
2  const User = db.User;
3
4  // =====
5  // GET ALL USERS
6  // =====
7
8  exports.getAllUsers = async (req, res) => {
9    try {
10     const users = await User.findAll({
11       order: [["created_at", "DESC"]],
12     });
13
14     res.json({
15       success: true,
16       count: users.length,
17       data: users,
18     });
19   } catch (error) {
20     console.error("Error fetching users:", error);
21
22     res.status(500).json({
23       success: false,
24       message: "Failed to fetch users",
25       error: error.message,
26     });
27   }
28 };
29
30 // =====
31 // GET USER BY ID
```

Penjelasan

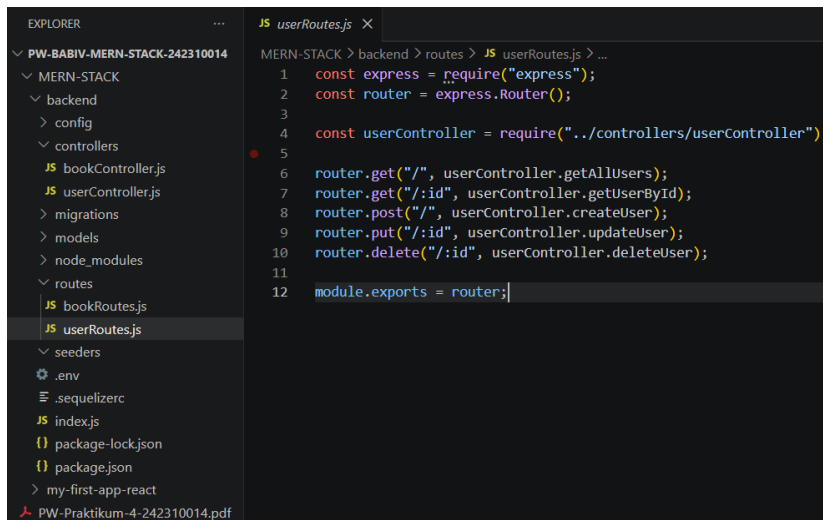
Controller bertugas menerima request dari client, melakukan proses terhadap database menggunakan model Sequelize, kemudian mengirimkan response dalam format JSON.

6. Implementasi Routing

Selanjutnya dibuat file **userRoutes.js** untuk mendefinisikan endpoint REST API.

Endpoint yang digunakan yaitu:

Method	Endpoint	Fungsi
GET	/api/users	Menampilkan seluruh data user
GET	/api/users/:id	Menampilkan data user berdasarkan ID
POST	/api/users	Menambahkan user baru
PUT	/api/users/:id	Mengubah data user
DELETE	/api/users/:id	Menghapus data user



```
1 const express = require("express");
2 const router = express.Router();
3
4 const userController = require("../controllers/userController");
5
6 router.get("/", userController.getAllUsers);
7 router.get("/:id", userController.getUserById);
8 router.post("/", userController.createUser);
9 router.put("/:id", userController.updateUser);
10 router.delete("/:id", userController.deleteUser);
11
12 module.exports = router;
```

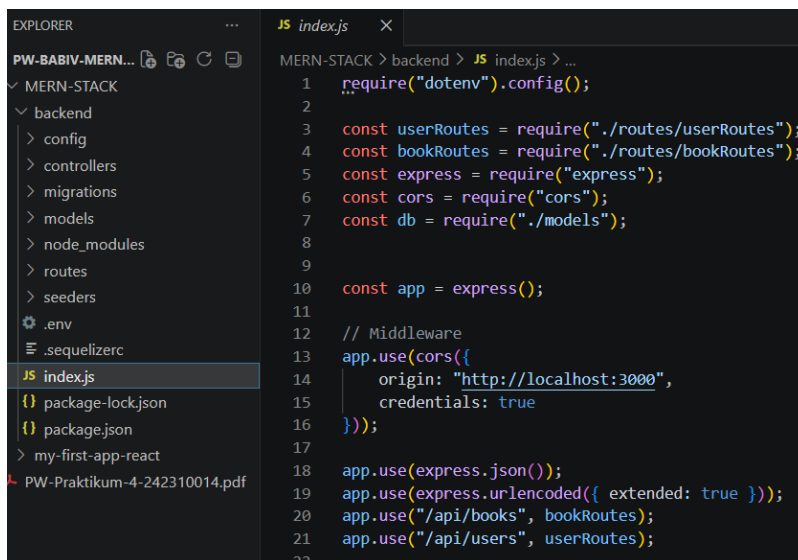
Penjelasan

Routing berfungsi menghubungkan setiap URL yang diakses client dengan fungsi controller yang sesuai sehingga setiap request dapat diproses dengan benar.

7. Integrasi Route ke Express

Agar endpoint dapat digunakan, file **userRoutes.js** didaftarkan pada **index.js**.

```
app.use("/api/users", userRoutes);
```



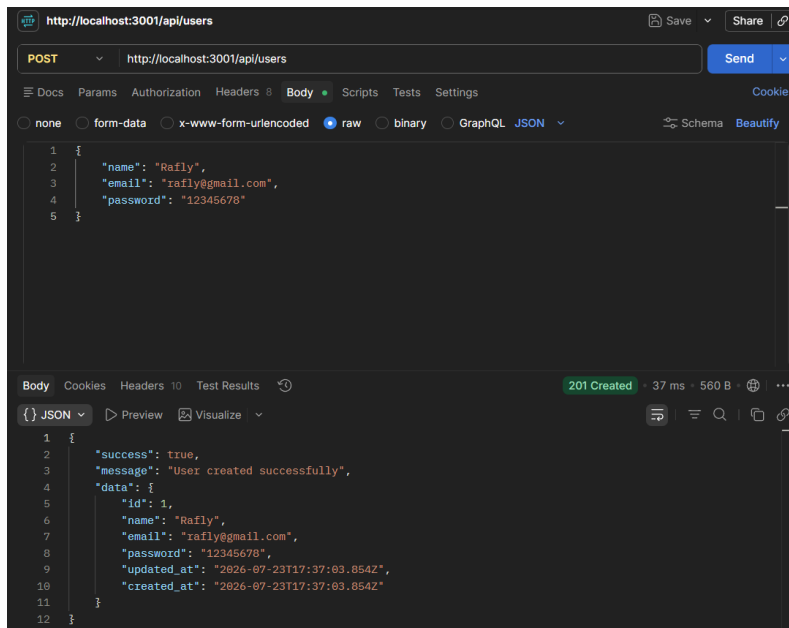
```
1 require("dotenv").config();
2
3 const userRoutes = require("../routes/userRoutes");
4 const bookRoutes = require("../routes/bookRoutes");
5 const express = require("express");
6 const cors = require("cors");
7 const db = require("../models");
8
9
10 const app = express();
11
12 // Middleware
13 app.use(cors({
14   origin: "http://localhost:3000",
15   credentials: true
16 }));
17
18 app.use(express.json());
19 app.use(express.urlencoded({ extended: true }));
20 app.use("/api/books", bookRoutes);
21 app.use("/api/users", userRoutes);
22
```

Penjelasan

Penambahan route ini membuat seluruh endpoint User Management dapat diakses menggunakan prefix **/api/users**.

8. Pengujian REST API

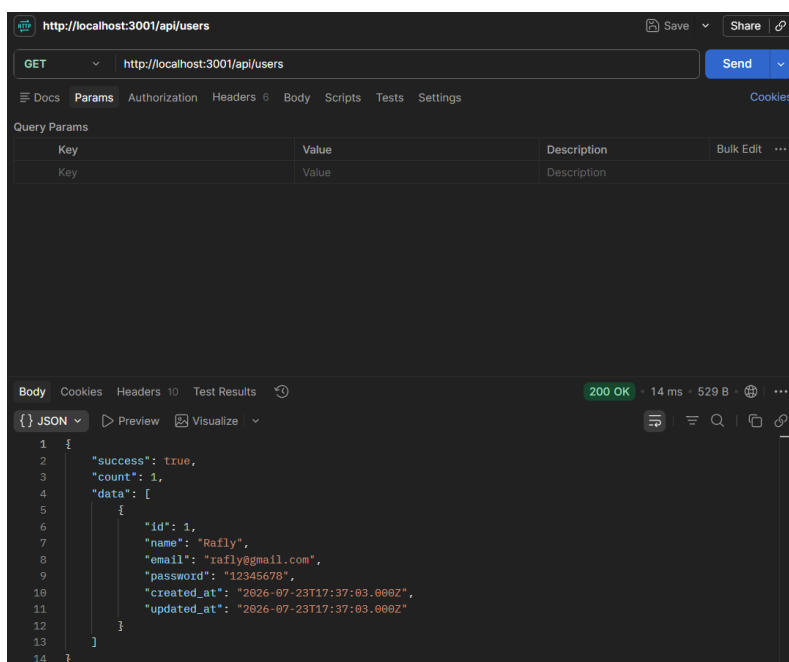
a. Create User



Penjelasan

Pengujian berhasil dilakukan dengan status **201 Created** yang menunjukkan data pengguna baru berhasil disimpan ke database.

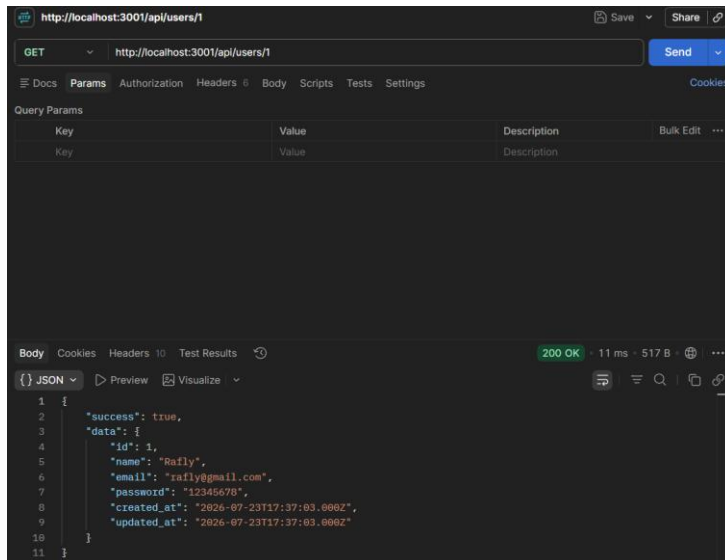
b. Get All User



Penjelasan

Endpoint berhasil menampilkan seluruh data pengguna yang tersimpan pada database beserta jumlah data yang tersedia.

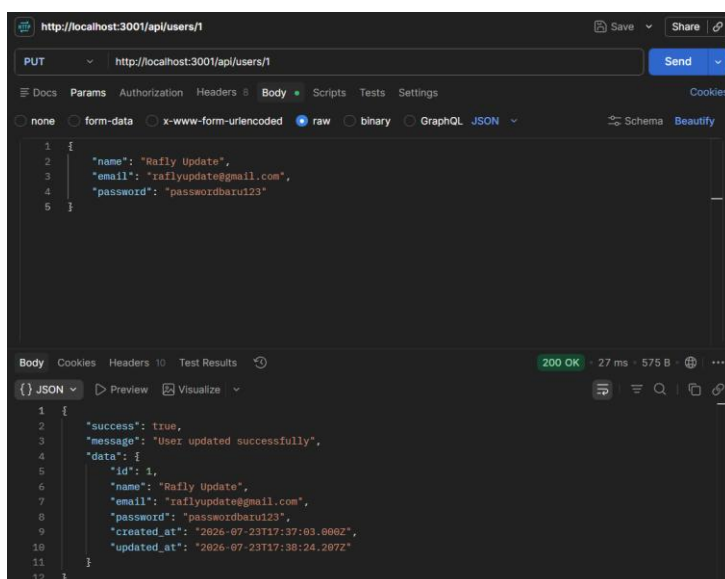
c. Get User By ID



Penjelasan

Pengujian berhasil menampilkan informasi pengguna berdasarkan ID yang diminta.

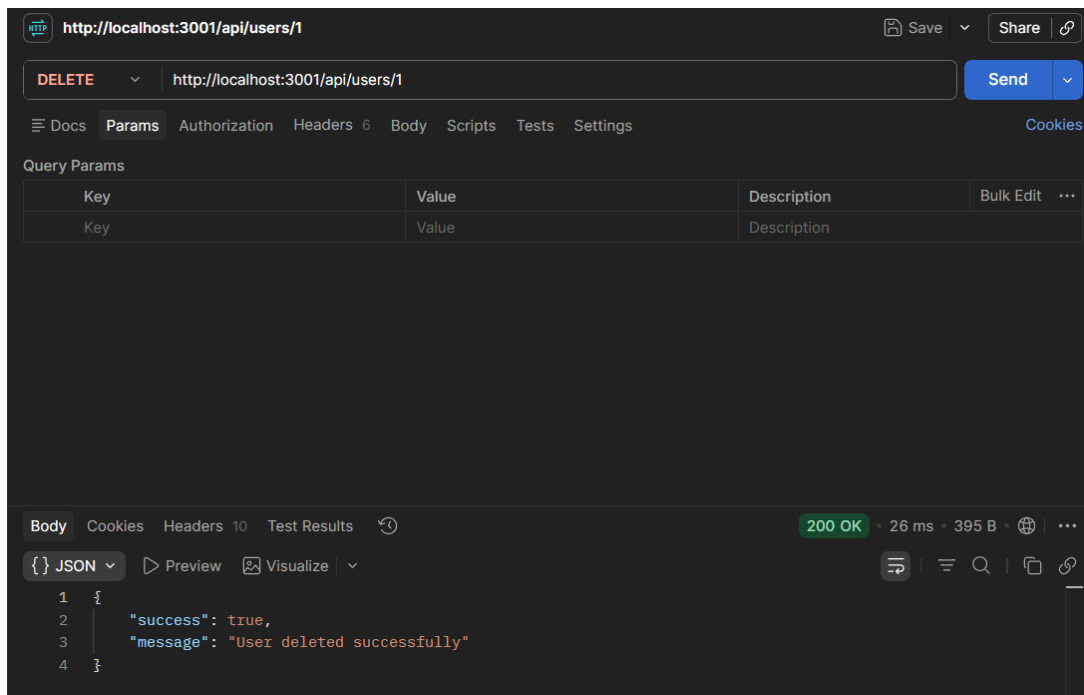
d. Update User



Penjelasan

Data pengguna berhasil diperbarui sesuai dengan informasi baru yang dikirimkan melalui request.

e. Delete User



Penjelasan

Data pengguna berhasil dihapus dari database dan sistem mengembalikan pesan bahwa proses penghapusan berhasil dilakukan.

9. Kesimpulan

Berdasarkan hasil implementasi dan pengujian, modul **User Management** berhasil dibangun menggunakan **Node.js**, **Express.js**, dan **Sequelize ORM**. Seluruh endpoint REST API yang terdiri dari Create, Read, Update, dan Delete dapat berjalan dengan baik serta menghasilkan status HTTP yang sesuai. Migration berhasil membuat tabel **users**, sedangkan model, controller, dan routing berhasil saling terintegrasi sehingga proses pengelolaan data pengguna dapat dilakukan secara optimal. Dengan demikian, seluruh tujuan pada tugas pembelajaran berhasil dicapai sesuai dengan ketentuan praktikum.